



Facultad de Ingeniería
Ingeniería de Sistemas e Informática

Trabajo Final:

Sistema web para el control financiero y análisis de riesgo en casas de apuestas

Nombre de los Estudiantes: Girao Breña Pablo Humberto

Asignatura: Lenguajes de Programación

Docente: CARLOS ALFONSO OLIVARES CASAPIA

Lima – Perú
2026

Índice

Objetivos	¡Error! Marcador no definido.
Objetivo general	¡Error! Marcador no definido.
Objetivos específicos.....	¡Error! Marcador no definido.
Proceso para automatizar	¡Error! Marcador no definido.
Flujo del proceso automatizado	¡Error! Marcador no definido.
Entradas, procesamiento y salidas	¡Error! Marcador no definido.
Diagrama de actividades del proceso	¡Error! Marcador no definido.
Metodología de desarrollo.....	¡Error! Marcador no definido.
CAPÍTULO I: CONTEXTO Y ALCANCE DEL PROYECTO	¡Error! Marcador no definido.
1.1 Informe situacional de la solución	¡Error! Marcador no definido.
1.1.1 Alcance del negocio.....	¡Error! Marcador no definido.
1.1.2 Ubicación e historia de la iniciativa	¡Error! Marcador no definido.
1.1.3 Descripción de la operación	¡Error! Marcador no definido.
CAPÍTULO II: SITUACIÓN PROBLEMÁTICA Y PROPUESTA DE AUTOMATIZACIÓN	¡Error! Marcador no definido.
2.1 Situación problemática	¡Error! Marcador no definido.
2.1.1 Opacidad transaccional.....	¡Error! Marcador no definido.
2.1.2 Congelamiento asimétrico por rollover	¡Error! Marcador no definido.
2.1.3 Sesgo de ilusión de ganancia	¡Error! Marcador no definido.
2.1.4 Otros riesgos identificados	¡Error! Marcador no definido.
2.2 Proceso a solucionar mediante automatización	¡Error! Marcador no definido.
2.2.1 Flujo automatizado	¡Error! Marcador no definido.
CAPÍTULO III: MODELAMIENTO ORIENTADO A OBJETOS ...	¡Error! Marcador no definido.
3.1 Descripción de componentes POO.....	¡Error! Marcador no definido.
3.1.1 Modelo conceptual.....	¡Error! Marcador no definido.
3.2 Modelamiento UML	¡Error! Marcador no definido.
3.2.1 UML conceptual del proyecto	¡Error! Marcador no definido.
3.2.2 UML de trazabilidad de la implementación final	¡Error! Marcador no definido.

Índice

Objetivos	5
Objetivo general	5
Objetivos específicos.....	5
Proceso para automatizar	5
Flujo del proceso automatizado	6
Entradas, procesamiento y salidas	7
Diagrama de actividades del proceso	8
Metodología de desarrollo.....	9
CAPÍTULO I: CONTEXTO Y ALCANCE DEL PROYECTO	10
1.1 Informe situacional de la solución	10
1.1.1 Alcance del negocio.....	10
1.1.2 Ubicación e historia de la iniciativa	11
1.1.3 Descripción de la operación	11
CAPÍTULO II: SITUACIÓN PROBLEMÁTICA Y PROPUESTA DE AUTOMATIZACIÓN	13
2.1 Situación problemática	13
2.1.1 Opacidad transaccional.....	13
2.1.2 Congelamiento asimétrico por rollover	13
2.1.3 Sesgo de ilusión de ganancia	14
2.1.4 Otros riesgos identificados	14
2.2 Proceso a solucionar mediante automatización	15
2.2.1 Flujo automatizado	15
CAPÍTULO III: MODELAMIENTO ORIENTADO A OBJETOS	18
3.1 Descripción de componentes POO.....	18
3.1.1 Modelo conceptual.....	18
3.2 Modelamiento UML	19
3.2.1 UML conceptual del proyecto.....	19
3.2.2 UML de trazabilidad de la implementación final	20
CAPÍTULO IV: DISEÑO DE LA BASE DE DATOS RELACIONAL	21
4.1 Modelo de base de datos relacional	21
4.1.1 Enfoque general	21
4.1.2 Entidades del esquema final	21

4.1.3 Relación central del dominio	22
4.1.4 Modelo entidad-relación.....	23
4.2 Implementación de la base de datos en MySQL	23
4.3 Esquema en MySQL	27
CAPÍTULO V: ARQUITECTURA Y DESARROLLO DE LA SOLUCIÓN.....	28
5.1 Arquitectura de desarrollo de la solución	28
5.1.1 Vista lógica de tres capas	28
5.1.2 Arquitectura física	29
5.1.3 Flujo de datos	31

Objetivos

Objetivo general

Desarrollar una aplicación web multiusuario que permita registrar, auditar y analizar operaciones realizadas en casas de apuestas, diferenciando saldos propios, bonos, fondos retirables y obligaciones de rollover, con el propósito de generar indicadores y alertas automáticas de riesgo.

Objetivos específicos

1. Implementar un registro transaccional ordenado para depósitos, bonos, apuestas, liquidaciones, cash out y retiros.
2. Mantener billeteras independientes por usuario y casa de apuestas.
3. Aplicar reglas de negocio para actualizar saldos y calcular el rollover pendiente.
4. Construir indicadores de rentabilidad, exposición, drawdown (caída máxima), concentración y comportamiento mediante Pandas y NumPy.
5. Emitir mensajes automáticos cuando se detecten patrones financieros o conductuales de riesgo.
6. Proteger las credenciales con Bcrypt y conservar sesiones persistentes sin almacenar contraseñas ni tokens en texto plano.
7. Garantizar integridad referencial, transaccionalidad y aislamiento multiusuario en MySQL.
8. Evidenciar los fundamentos de programación y POO exigidos por el sílabo del curso.

Proceso para automatizar

El proceso seleccionado para la automatización es el registro, conciliación y análisis financiero de las operaciones que un usuario realiza en diferentes casas de apuestas. En un escenario manual, la persona debe consultar historiales separados, distinguir entre dinero depositado y bonos promocionales, calcular el

rollover pendiente, reconstruir ganancias o pérdidas y comparar resultados entre plataformas. Esta labor demanda tiempo, es susceptible a errores aritméticos y puede producir una percepción equivocada sobre la disponibilidad real del capital.

La aplicación automatiza el proceso desde la recepción de la información hasta la presentación de indicadores y alertas. Cada operación se vincula con el usuario autenticado y con la casa seleccionada, lo que garantiza que los movimientos, saldos y resultados permanezcan separados entre cuentas.

Flujo del proceso automatizado

1. **Autenticación del usuario.** El sistema valida las credenciales mediante Bcrypt o recupera una sesión vigente. Si la autenticación no es válida, se impide el acceso a la información financiera.
2. **Selección de la operación.** El usuario elige registrar una recarga, bono, apuesta, liquidación, cash out o retiro.
3. **Ingreso de datos.** La interfaz Streamlit recibe la casa, el monto, la cuota, el tipo de saldo y los demás datos requeridos por la operación.
4. **Normalización y validación.** El sistema convierte textos y valores numéricos a los tipos correspondientes, comprueba campos obligatorios, montos positivos, cuotas válidas, límites personales y disponibilidad de saldo.
5. **Aplicación de reglas de negocio.** Según la operación, se actualizan el saldo depositado, saldo de bono, saldo retirable y rollover. Una apuesta pendiente se registra como capital expuesto; una liquidación determina el retorno y libera el rollover aplicable.
6. **Persistencia transaccional.** Los cambios se almacenan en MySQL mediante consultas parametrizadas. Si toda la operación finaliza correctamente, se ejecuta commit; ante una excepción, se ejecuta rollback para evitar datos parciales.
7. **Construcción analítica.** Las consultas se convierten en estructuras intermedias y DataFrames de Pandas. NumPy calcula totales, medias, ROI

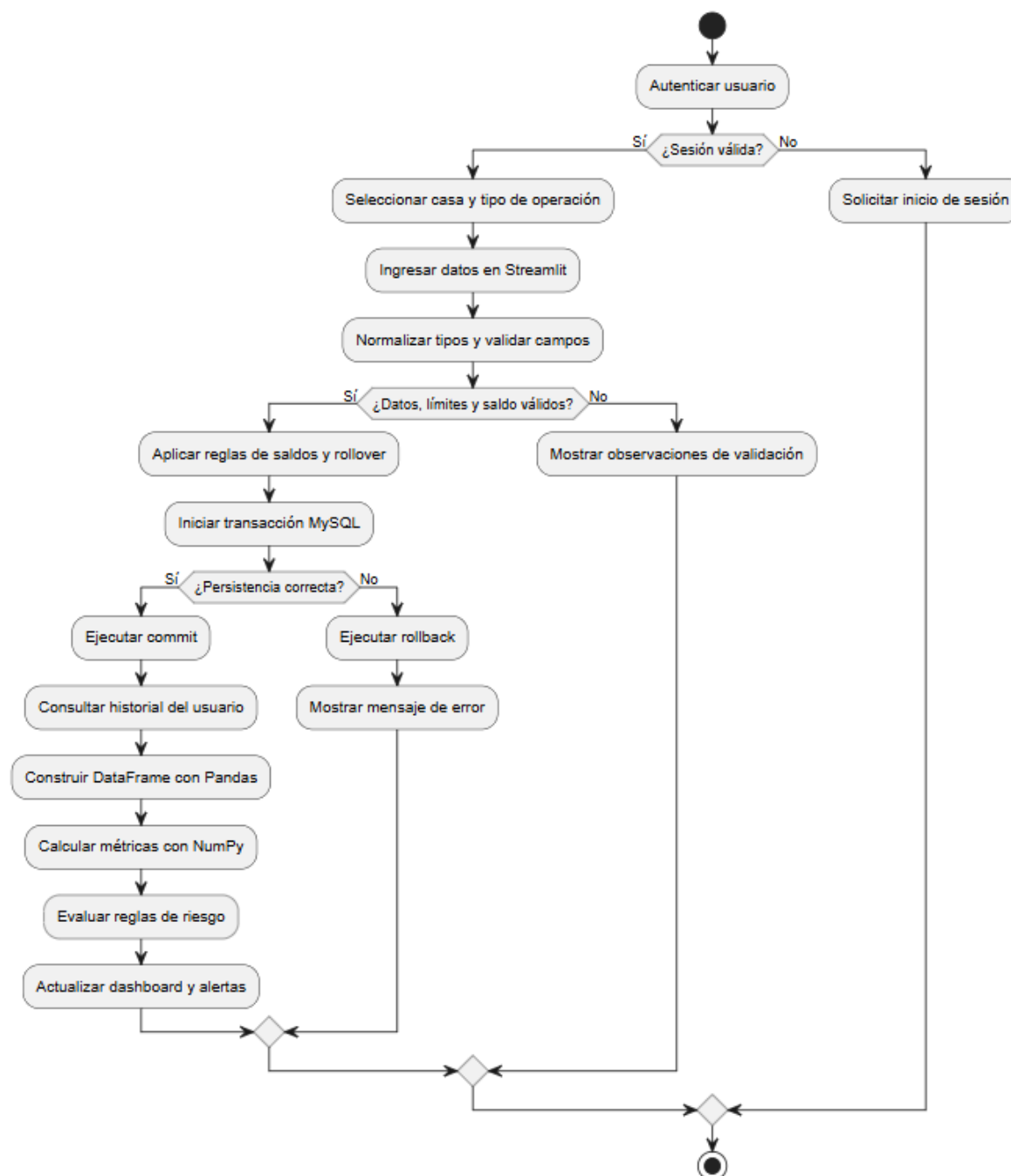
(Retorno de la Inversión), drawdown, concentración, rachas y otras métricas vectorizadas.

8. **Evaluación del riesgo.** Los indicadores se comparan con reglas preventivas para detectar descapitalización, ilusión de ganancia, persecución de pérdidas, concentración excesiva y rollover pendiente.
9. **Presentación del resultado.** El dashboard muestra saldos, movimientos, métricas y mensajes de alerta según su nivel de severidad.

Entradas, procesamiento y salidas

Etapa	Elementos principales
Entradas	Usuario autenticado, casa, tipo de movimiento, monto, cuota, saldo utilizado, evento y estado
Procesamiento transaccional	Validación, conciliación de billeteras, cálculo del retorno, actualización del rollover y control de límites
Procesamiento analítico	DataFrames, agregaciones, vectores NumPy, ROI, drawdown, concentración y patrones de comportamiento
Salidas	Saldos actualizados, historial auditable, dashboard individual y alertas automáticas de riesgo

Diagrama de actividades del proceso



El flujo es cíclico: después de registrar una operación, el usuario puede ingresar una nueva transacción y el sistema recalcula automáticamente su posición consolidada. No se requiere intervención manual para actualizar las métricas ni para decidir qué alertas deben mostrarse.

Metodología de desarrollo

El proyecto adopta una metodología incremental y orientada a componentes. En una primera etapa se identificó la problemática y se modelaron las entidades del dominio. En una segunda etapa se construyeron la persistencia relacional, las reglas transaccionales y la interfaz. Finalmente, se incorporaron autenticación, aislamiento multiusuario, analítica vectorizada, alertas y pruebas de integración.

El trabajo se organizó en cinco actividades:

1. **Análisis:** delimitación del problema, actores, operaciones y reglas de negocio.
2. **Diseño:** elaboración del modelo orientado a objetos, esquema relacional y arquitectura por capas.
3. **Implementación:** desarrollo modular de vistas, controladores, entidades y repositorios.
4. **Validación:** ejecución de pruebas funcionales, transaccionales y de aislamiento entre usuarios.

La metodología es aplicada, porque produce un software funcional; descriptiva, porque caracteriza el comportamiento financiero registrado; e incremental, porque cada avance incorpora nuevas capacidades sobre una base previamente validada.

CAPÍTULO I: CONTEXTO Y ALCANCE DEL PROYECTO

1.1 Informe situacional de la solución

1.1.1 Alcance del negocio

La propuesta se formula como Asesoría y Auditoría de Inversiones Alternativas de Alto Riesgo, orientada al consumidor de entretenimiento digital de Lima Metropolitana. Su producto central es una plataforma web que permite reconstruir y analizar la posición financiera real del usuario frente a distintas casas de apuestas.

El alcance funcional comprende:

1. Registro y autenticación individual de usuarios.
2. Configuración independiente de casas de apuestas.
3. Registro de depósitos, bonos, apuestas, cash out y retiros.
4. Separación contable entre saldo depositado, saldo de bono y saldo retirable.
5. Cálculo y seguimiento de rollover por casa.
6. Registro de apuestas pendientes y de su exposición potencial.
7. Cálculo de resultados, ROI, drawdown, tasa de acierto y concentración de capital.
8. Detección de patrones conductuales de riesgo.
9. Emisión automática de advertencias y recomendaciones conservadoras.
10. Aislamiento total de datos entre usuarios.

El producto no se conecta directamente con las plataformas de apuestas ni ejecuta operaciones monetarias. La información es ingresada por el propio usuario y se utiliza con fines de control, trazabilidad y educación financiera preventiva.

1.1.2 Ubicación e historia de la iniciativa

La iniciativa se sitúa teóricamente en Lima, Perú, como respuesta al crecimiento acelerado de las plataformas de apuestas y entretenimiento digital. La disponibilidad permanente de estas aplicaciones ha incrementado la frecuencia con la que los usuarios realizan depósitos, reciben promociones, aceptan bonos y ejecutan apuestas sin disponer necesariamente de un sistema independiente de control.

La idea de negocio surge al observar tres carencias:

- Los historiales proporcionados por las casas priorizan el detalle operativo, pero no siempre presentan una lectura consolidada del capital propio.
- Los bonos pueden incrementar el saldo visible sin representar dinero inmediatamente retirable.
- La evaluación intuitiva del usuario suele concentrarse en apuestas ganadas de manera aislada y no en el resultado neto total.

Frente a ello, se propone una capa independiente de auditoría personal que centraliza información, manteniendo a la vez la posibilidad de agregar otras casas con sus propias condiciones.

1.1.3 Descripción de la operación

La operación del servicio se basa en un ciclo de auditoría continua:

1. El usuario crea una cuenta protegida mediante contraseña cifrada con Bcrypt.
2. La aplicación crea para ese usuario las casas predeterminadas y sus billeteras lógicas.
3. El usuario registra depósitos y bonos recibidos.
4. El sistema calcula el rollover generado de acuerdo con la regla particular de la casa.

5. El usuario registra apuestas indicando monto, cuota, origen del saldo y estado.
6. La aplicación concilia el capital utilizado con las billeteras registradas.
7. Las apuestas pendientes se mantienen como exposición sin considerarse todavía ganancia o pérdida realizada.
8. Al liquidar una apuesta, se actualizan saldos, retornos y rollover.
9. Pandas estructura el historial y NumPy calcula métricas vectorizadas.
10. El dashboard presenta resultados, alertas, exposición y disponibilidad de retiro.

La propuesta de valor consiste en mostrar la diferencia entre saldo visible, dinero propio, bono condicionado y capital realmente retirable.

CAPÍTULO II: SITUACIÓN PROBLEMÁTICA Y PROPUESTA DE AUTOMATIZACIÓN

2.1 Situación problemática

2.1.1 Opacidad transaccional

El usuario puede mantener actividad simultánea en varias plataformas. Cada una presenta sus movimientos con formatos, conceptos y reglas particulares. Como consecuencia, la persona no dispone de una visión consolidada de cuánto depositó, cuánto retiró, cuánto mantiene expuesto y cuál es su resultado neto real.

La fragmentación de la información produce una opacidad práctica: los datos existen, pero no se transforman automáticamente en una explicación financiera comprensible. El usuario debe reconstruir manualmente el historial, distinguir capital propio de fondos promocionales y comparar movimientos entre plataformas.

2.1.2 Congelamiento asimétrico por rollover

El rollover es una condición que obliga a apostar un volumen determinado antes de habilitar el retiro de ciertos fondos. Esta regla puede afectar depósitos, bonos o ambos, dependiendo de la casa.

El problema se vuelve asimétrico cuando:

- El usuario percibe el saldo como disponible, aunque esté condicionado.
- Una nueva recarga incrementa nuevamente el rollover pendiente.
- El bono eleva el saldo nominal, pero no necesariamente el dinero propio.
- La cuota de una apuesta puede no ser suficiente para liberar rollover.
- El usuario intenta recuperar pérdidas mediante nuevas exposiciones.

La solución separa el saldo de depósito, el bono y el saldo retirable; además, muestra el rollover generado, liberado y pendiente por casa.

2.1.3 Sesgo de ilusión de ganancia

El sesgo de ilusión de ganancia ocurre cuando el usuario interpreta una alta cantidad de aciertos como evidencia de rentabilidad. Es posible ganar varias apuestas pequeñas y perder una apuesta de mayor monto, obteniendo una tasa de acierto alta y, al mismo tiempo, un balance negativo.

Ejemplo conceptual:

- Ganancia 1: S/ 4.00.
- Ganancia 2: S/ 5.00.
- Ganancia 3: S/ 3.00.
- Pérdida posterior: S/ 30.00.
- Resultado neto: pérdida de S/ 18.00.

Aunque existan tres decisiones ganadoras y una perdedora, el capital se ha reducido. El sistema detecta este patrón comparando la tasa de acierto con el flujo efectivo acumulado.

2.1.4 Otros riesgos identificados

- Persecución de pérdidas mediante incremento del monto apostado.
- Rachas consecutivas de resultados negativos.
- Drawdown elevado respecto del capital expuesto.
- Concentración de la mayor parte del saldo en una sola casa.
- Bonos que camuflan un flujo de caja negativo.
- Apuestas registradas sin respaldo completo en las billeteras.
- Confusión entre retorno total y ganancia neta.
- Confusión entre saldo para jugar y saldo retirable.

2.2 Proceso a solucionar mediante automatización

2.2.1 Flujo automatizado

Etapas 1: recepción de datos

La capa de presentación muestra formularios Streamlit para registrar:

- Casa de apuestas.
- Monto depositado.
- Bono recibido.
- Monto apostado.
- Cuota decimal.
- Origen del saldo.
- Deporte, liga, evento y detalle.
- Fecha del evento.
- Resultado o estado pendiente.
- Monto de cash out.
- Retiro solicitado.

Etapas 2: parsing, normalización y conversión

Los valores ingresados se convierten a tipos concretos de Python:

- Identificadores y descripciones: str.
- Identificador de usuario y apuesta: int.
- Montos, cuotas, saldos y métricas: float durante el cálculo.
- Confirmaciones y estados de control: bool.

Los textos son limpiados con operaciones como eliminación de espacios, conversión a mayúsculas y separación por comas. Las fechas se normalizan al formato ISO AAAA-MM-DD.

Etapas 3: validación de la entidad

La información de una apuesta es convertida en un objeto RegistroApuesta. El constructor valida que el monto sea positivo, que exista una casa, que la cuota sea mayor que 1.00, que el tipo de saldo sea válido y que los campos deportivos estén completos.

Etapas 4: aplicación de reglas financieras

El backend ejecuta, entre otras, las siguientes reglas:

- El depósito aumenta la billetera de depósito.
- El bono aumenta exclusivamente la billetera promocional.
- Cada recarga genera rollover según los multiplicadores de la casa.
- Una apuesta pendiente reserva exposición, pero no se considera resultado realizado.
- Una apuesta perdida descuenta el capital conciliado.
- Una apuesta ganada acredita el retorno al saldo retirable.
- Cuando se apuesta con bono, solo la ganancia neta se transforma en retirable.
- Una apuesta anulada restituye el monto conciliado.
- El cash out registra exactamente el monto recibido.
- El retiro exige saldo suficiente, mínimo alcanzado y rollover igual a cero.

Etapas 5: persistencia transaccional

Las operaciones se ejecutan dentro de context managers de conexión. Si todas las instrucciones terminan correctamente, se confirma la transacción mediante commit. Si ocurre una excepción, se ejecuta rollback, evitando actualizaciones parciales de saldos e historial.

Etapas 6: construcción analítica

Las consultas MySQL se transforman en listas de diccionarios y posteriormente en DataFrames. Pandas limpia fechas, descarta registros incompletos, ordena cronológicamente y agrupa resultados por casa.

NumPy convierte las columnas numéricas en arreglos y calcula sumas, medias, mínimos, máximos, acumulados, máscaras booleanas, ROI, drawdown y concentración.

Etapas 7: activación de alertas

Los resultados estadísticos se contrastan con umbrales de riesgo. Cuando se cumple una condición, se agrega un mensaje estructurado con código, nivel y descripción. Streamlit representa las alertas como mensajes informativos, advertencias o errores según su severidad.

CAPÍTULO III: MODELAMIENTO ORIENTADO A OBJETOS

3.1 Descripción de componentes POO

3.1.1 Modelo conceptual

El modelo conceptual solicitado considera tres clases principales.

Clase padre Transaccion

Representa una operación financiera genérica asociada a una casa.

Elemento	Tipo	Finalidad
id_casa	str	Identifica la casa de apuestas.
monto	float	Representa el importe de la operación.
tipo_saldo	str	Indica depósito, bono o saldo retirable.
__init__()	Constructor	Normaliza atributos y valida el monto.

Clase hija conceptual RegistroApuesta

Especializa una transacción para representar una apuesta.

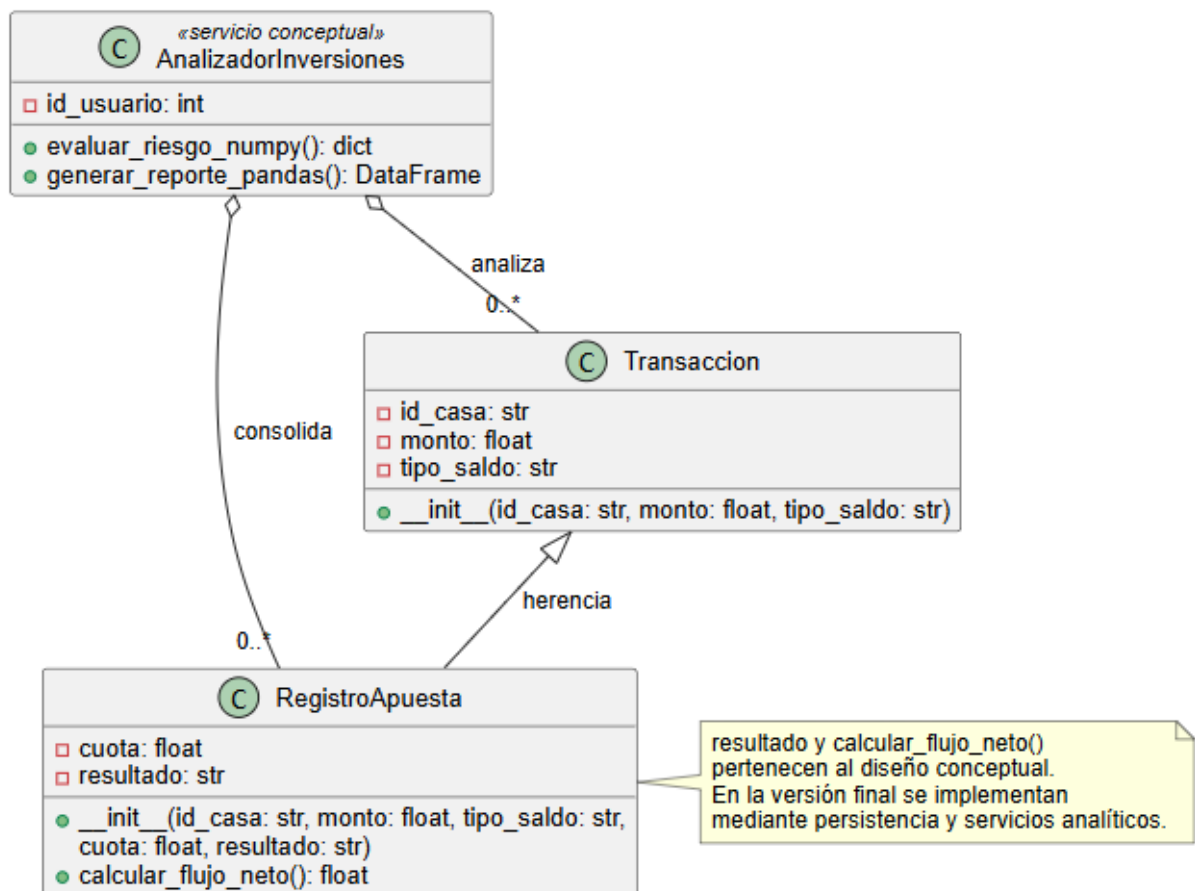
Elemento	Tipo	Finalidad
cuota	float	Cuota decimal de la apuesta.
resultado	str	Resultado conceptual: pendiente, ganada, perdida, anulada o cash out.
calcular_flujo_neto()	Método conceptual	Calcula ganancia neta o pérdida según monto, cuota y resultado.

Servicio conceptual AnalizadorInversiones

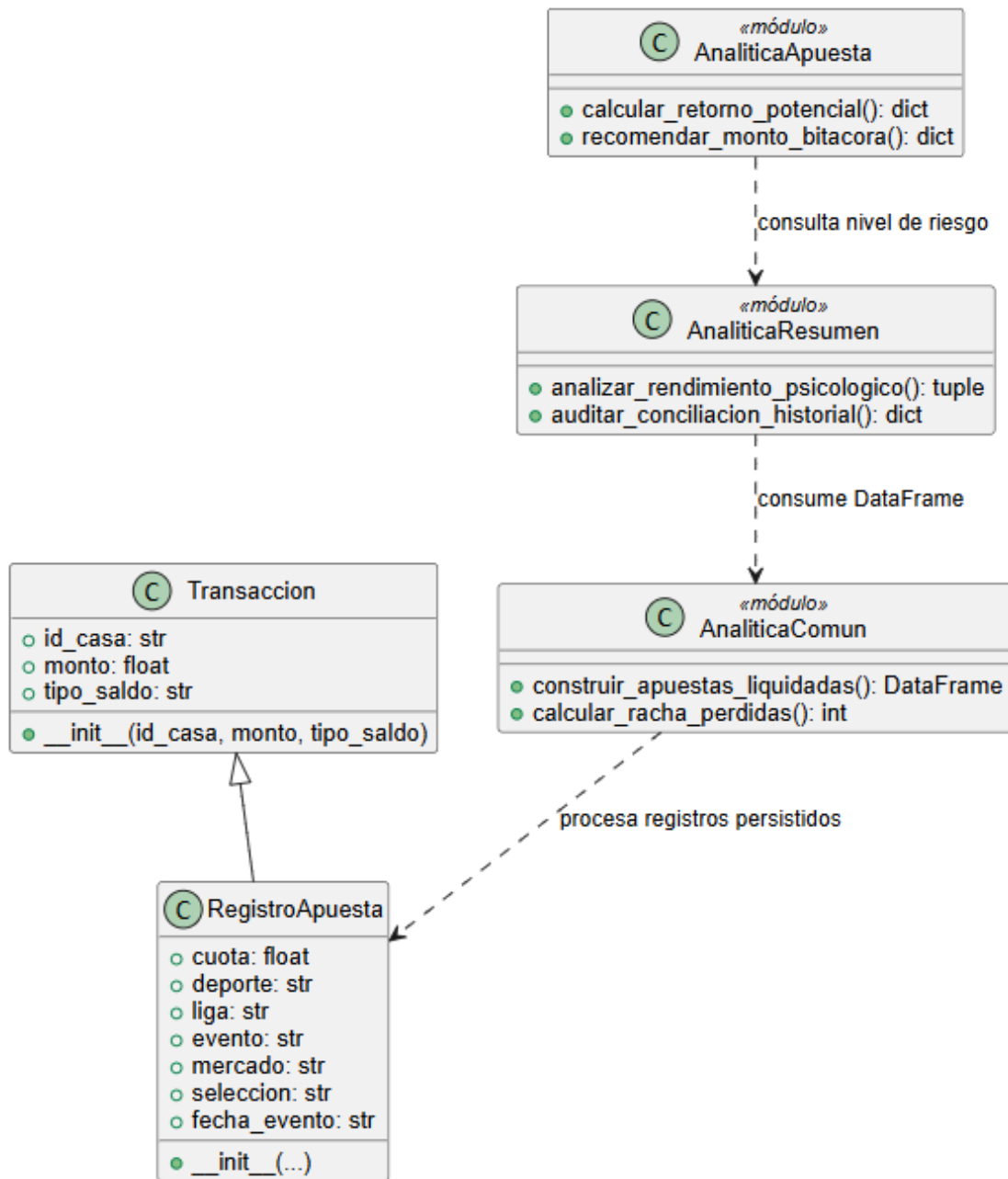
Elemento	Tipo	Finalidad
id_usuario	int	Delimita los datos que pueden ser analizados.
evaluar_riesgo_numpy()	Método conceptual	Ejecuta métricas vectorizadas y reglas de alerta.
generar_reporte_pandas()	Método conceptual	Construye el reporte tabular del usuario.

3.2 Modelamiento UML

3.2.1 UML conceptual del proyecto



3.2.2 UML de trazabilidad de la implementación final



CAPÍTULO IV: DISEÑO DE LA BASE DE DATOS RELACIONAL

4.1 Modelo de base de datos relacional

4.1.1 Enfoque general

La solución utiliza una única base MySQL multiusuario. Todas las entidades financieras incorporan `usuario_id`, lo que permite relacionarlas con la cuenta propietaria y aplicar filtros explícitos en cada consulta.

El modelo evita que dos usuarios compartan accidentalmente configuraciones, saldos o movimientos. Una casa se identifica mediante la clave compuesta (`usuario_id`, `id`), por lo que dos usuarios pueden tener una casa denominada BETANO sin colisión.

4.1.2 Entidades del esquema final

Usuarios:

Almacena identidad, correo único, hash Bcrypt y estado de la cuenta. Es la entidad principal de propiedad.

sesiones_usuario:

Almacena exclusivamente el hash SHA-256 de cada token de sesión, su fecha de creación y expiración. La eliminación del usuario elimina sus sesiones mediante ON DELETE CASCADE.

casas_apuestas:

Conserva las reglas y billeteras independientes de cada usuario:

- Saldo de depósito.
- Saldo de bono.
- Saldo retirable.

- Rollover pendiente.
- Retiro mínimo.
- Cuota mínima válida.
- Deportes disponibles.

Su clave primaria compuesta es (usuario_id, id).

Apuestas:

Registra la información deportiva y financiera de cada apuesta. Se relaciona con la casa mediante (usuario_id, id_casa). Contiene monto, cuota, tipo de saldo, estado, retorno, capital conciliado, rollover liberado y fechas de registro y resolución.

historial_transacciones:

Funciona como bitácora auditable. Registra recargas, bonos, retiros, apuestas pendientes y liquidaciones. Cada movimiento se relaciona con el usuario, con una casa y, cuando corresponde, con una apuesta.

configuracion_control:

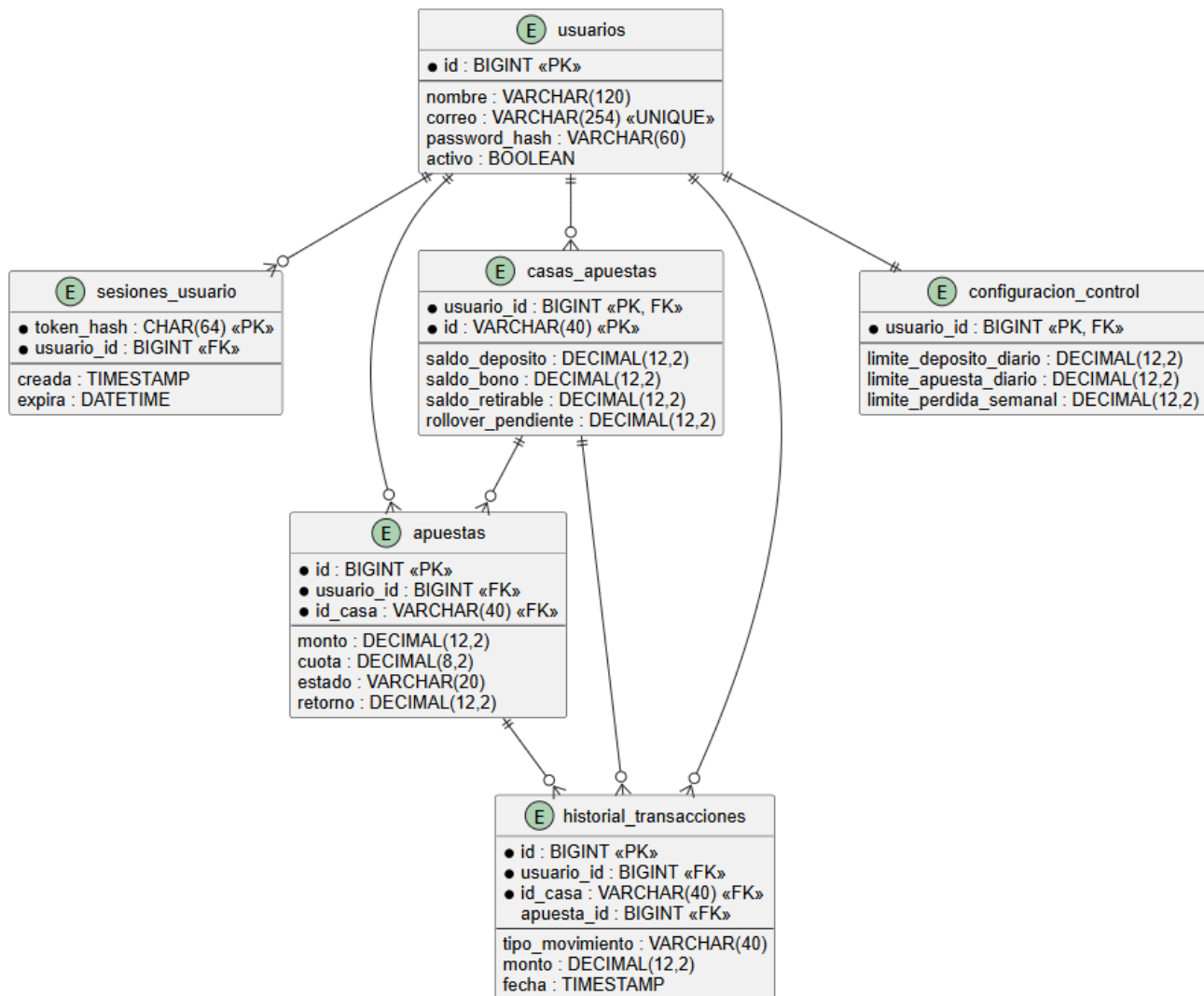
Mantiene límites personales por usuario: depósito diario, apuesta diaria, apuesta individual, pérdida semanal, pausa y modo estricto.

4.1.3 Relación central del dominio

Las dos tablas financieras centrales son casas_apuestas e historial_transacciones:

- Una casa pertenece a un usuario.
- Una casa puede tener muchos movimientos.
- Un movimiento pertenece exactamente a un usuario y una casa.
- La FK compuesta impide asociar un movimiento con una casa de otro usuario.
- Si se elimina un usuario, sus casas desaparecen en cascada.
- Si una apuesta es eliminada, sus movimientos asociados se eliminan en cascada.

4.1.4 Modelo entidad-relación



4.2 Implementación de la base de datos en MySQL

El siguiente script corresponde al esquema relacional de la aplicación final. Se utiliza DECIMAL para evitar errores de representación binaria en importes financieros, InnoDB para soportar transacciones y llaves foráneas, y ON DELETE CASCADE cuando la dependencia debe desaparecer con su propietario.


```
CREATE DATABASE IF NOT EXISTS apuestas_analitica_multiusuario
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE apuestas_analitica_multiusuario;
```

-- Identidad y autenticación principal.

```
CREATE TABLE IF NOT EXISTS usuarios (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(120) NOT NULL,
  correo VARCHAR(254) NOT NULL UNIQUE,
  password_hash VARCHAR(60) NOT NULL,
  fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  activo BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;
```

-- Sesiones persistentes; nunca almacena el token original.

```
CREATE TABLE IF NOT EXISTS sesiones_usuario (
  token_hash CHAR(64) PRIMARY KEY,
  usuario_id BIGINT NOT NULL,
  creada TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expira DATETIME NOT NULL,
  CONSTRAINT fk_sesion_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuarios(id) ON DELETE CASCADE,
  INDEX idx_sesion_usuario (usuario_id)
) ENGINE=InnoDB;
```

-- Billeteras y reglas independientes por usuario y casa.

```
CREATE TABLE IF NOT EXISTS casas_apuestas (
  usuario_id BIGINT NOT NULL,
  id VARCHAR(40) NOT NULL,
  nombre_casa VARCHAR(120) NOT NULL,
  saldo_deposito DECIMAL(12,2) NOT NULL DEFAULT 0,
  saldo_bono DECIMAL(12,2) NOT NULL DEFAULT 0,
  saldo_retirable DECIMAL(12,2) NOT NULL DEFAULT 0,
  rollover_pendiente DECIMAL(12,2) NOT NULL DEFAULT 0,
```

```

minimo_retiro DECIMAL(12,2) NOT NULL DEFAULT 50,
rollover_deposito DECIMAL(8,2) NOT NULL DEFAULT 1,
rollover_bono DECIMAL(8,2) NOT NULL DEFAULT 1,
cuota_minima_rollover DECIMAL(8,2) NOT NULL DEFAULT 1.01,
deportes VARCHAR(500) NOT NULL DEFAULT 'Futbol,Baloncesto,Tenis',
PRIMARY KEY (usuario_id, id),
CONSTRAINT fk_casa_usuario FOREIGN KEY (usuario_id)
REFERENCES usuarios(id) ON DELETE CASCADE
) ENGINE=InnoDB;

```

-- Operaciones deportivas y conciliación de capital.

```

CREATE TABLE IF NOT EXISTS apuestas (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
usuario_id BIGINT NOT NULL,
id_casa VARCHAR(40) NOT NULL,
deporte VARCHAR(80) NOT NULL,
liga VARCHAR(120) NOT NULL,
evento VARCHAR(255) NOT NULL,
mercado VARCHAR(120) NOT NULL,
seleccion VARCHAR(255) NOT NULL,
fecha_evento DATE NOT NULL,
monto DECIMAL(12,2) NOT NULL CHECK (monto > 0),
cuota DECIMAL(8,2) NOT NULL CHECK (cuota > 1),
tipo_saldo VARCHAR(20) NOT NULL,
estado VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE',
retorno DECIMAL(12,2) NOT NULL DEFAULT 0,
monto_conciliado DECIMAL(12,2) NOT NULL DEFAULT 0,
rollover_liberado DECIMAL(12,2) NOT NULL DEFAULT 0,
fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
fecha_resolucion TIMESTAMP NULL,
UNIQUE KEY uq_apuesta_usuario_id (usuario_id, id),
CONSTRAINT fk_apuesta_usuario FOREIGN KEY (usuario_id)
REFERENCES usuarios(id) ON DELETE CASCADE,
CONSTRAINT fk_apuesta_casa FOREIGN KEY (usuario_id, id_casa)

```

```
REFERENCES casas_apuestas(usuario_id, id)
) ENGINE=InnoDB;
```

-- Libro de auditoría de movimientos financieros.

```
CREATE TABLE IF NOT EXISTS historial_transacciones (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  usuario_id BIGINT NOT NULL,
  id_casa VARCHAR(40) NOT NULL,
  apuesta_id BIGINT NULL,
  tipo_movimiento VARCHAR(40) NOT NULL,
  monto DECIMAL(12,2) NOT NULL,
  cuota DECIMAL(8,2) NOT NULL DEFAULT 1,
  tipo_saldo_usado VARCHAR(20) NOT NULL,
  fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_historial_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuarios(id) ON DELETE CASCADE,
  CONSTRAINT fk_historial_casa FOREIGN KEY (usuario_id, id_casa)
    REFERENCES casas_apuestas(usuario_id, id),
  CONSTRAINT fk_historial_apuesta FOREIGN KEY (usuario_id, apuesta_id)
    REFERENCES apuestas(usuario_id, id) ON DELETE CASCADE,
  INDEX idx_historial_usuario_fecha (usuario_id, fecha)
) ENGINE=InnoDB;
```

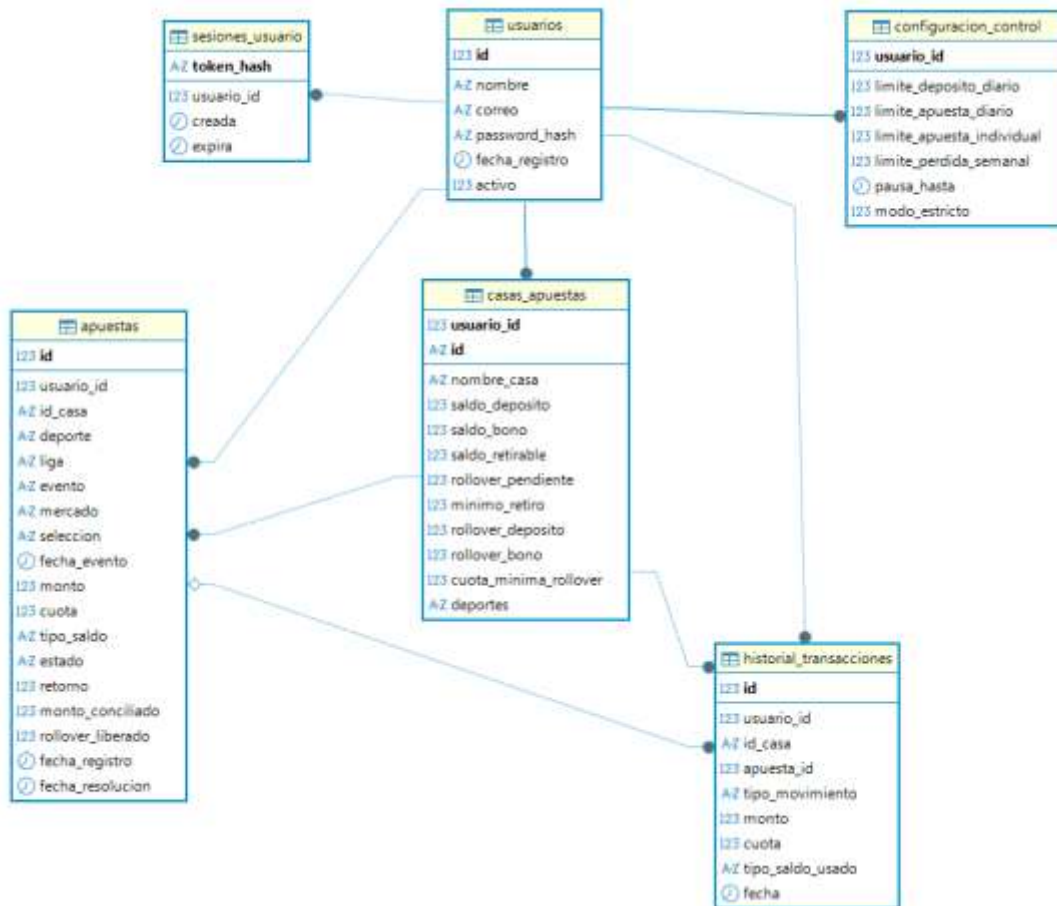
-- Parámetros preventivos personalizados.

```
CREATE TABLE IF NOT EXISTS configuracion_control (
  usuario_id BIGINT PRIMARY KEY,
  limite_deposito_diario DECIMAL(12,2) NOT NULL DEFAULT 100,
  limite_apuesta_diario DECIMAL(12,2) NOT NULL DEFAULT 100,
  limite_apuesta_individual DECIMAL(12,2) NOT NULL DEFAULT 20,
  limite_perdida_semanal DECIMAL(12,2) NOT NULL DEFAULT 150,
  pausa_hasta DATETIME NULL,
  modo_estricto BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT fk_configuracion_usuario FOREIGN KEY (usuario_id)
```

REFERENCES usuarios(id) ON DELETE CASCADE

) ENGINE=InnoDB;

4.3 Esquema en MySQL



CAPÍTULO V: ARQUITECTURA Y DESARROLLO DE LA SOLUCIÓN

5.1 Arquitectura de desarrollo de la solución

5.1.1 Vista lógica de tres capas

Capa de presentación

Implementada con Streamlit y ubicada en views/ y app.py. Sus responsabilidades son:

- Mostrar formularios y controles.
- Recibir información del usuario.
- Mostrar métricas, tablas, gráficos y alertas.
- Administrar la navegación y el estado visual.
- Solicitar casos de uso a Controllers.

La capa visual no contiene SQL. La validación de dominio de la apuesta se solicita mediante el controlador transaccional.

Capa de lógica de negocio y analítica

Ubicada en controllers/ y complementada por entidades de dominio. Incluye:

- Autenticación y validación de credenciales.
- Coordinación del registro de apuestas.
- Cálculo de retorno potencial.
- Cálculo de rollover estimado.
- Análisis de resultados.
- Recomendaciones conservadoras.
- Detección de patrones de riesgo.
- Construcción de resúmenes para el dashboard.

Pandas estructura la información; NumPy ejecuta cálculos vectorizados; y las entidades POO normalizan los datos antes de persistirlos.

Capa de persistencia

Ubicada en `models/` y `models/repositorios/`. Sus responsabilidades son:

- Crear conexiones y pools MySQL.
- Leer configuración desde `.env`. (datos sensibles se guardan en variable de entorno)
- Crear el esquema relacional.
- Ejecutar consultas parametrizadas.
- Aplicar filtros explícitos por usuario.
- Confirmar o revertir transacciones.
- Mantener la integridad referencial.

5.1.2 Arquitectura física

`bets_analitica/`

```
|— app.py
|— requirements.txt
|— .env
|— controllers/
|   |— autenticacion.py
|   |— transacciones.py
|   |— analitica_comun.py
|   |— analitica_resumen.py
|   |— analitica_apuesta.py
|   |— analitica_recarga.py
|   |— analitica_pendientes.py
|— models/
|   |— entidades.py
|   |— conexion_mysql.py
|   |— mysql_adapter.py
|   |— database.py
```

```
| |— usuarios.py
| |— repositorios/
| |   |— conexion.py
| |   |— casas.py
| |   |— apuestas.py
| |   |— movimientos.py
| |   |— historial.py
| |   |— configuracion.py
|— views/
| |— autenticacion.py
| |— componentes.py
| |— apuesta/
| |— casas/
| |— historial/
| |— pendientes/
| |— recarga/
| |— resumen/
| |— retiro/
```

5.1.3 Flujo de datos

